

个人答辩报告

12312420 王振宇

SUSTC

May 13, 2025

Content

- 1 负责工作简述
- 2 OutputModule
- 3 ClockDivider
- 4 SegGenerator
- 5 SegDisplay
- 6 ALU

负责工作简述

- ① OutputModule: CPU 输出模块，从寄存器中读取数据
 - ClockDivider: 时钟分频器
 - SegGenerator: 生成位选信号
 - SegDisplay: 生成段选信号，输出内容控制（2/10/16 进制）
- ② ALU: 算数逻辑单元

OutputModule

```

1 module OutputModule(
2     input clk,
3     input rst,
4     input en_output,
5     input [31:0] reg_a7,
6     input [31:0] output_data,
7     output [7:0] seg1,
8     output [7:0] seg2,
9     output [7:0] led,
10    output wire [7:0] an    //control
                                the 8-segment
11);
12
13 parameter PERIOD_SEG = 1<<16;
14
15 wire [31:0] val,tmp_reg_a7;
16 wire [3:0] select;
17 wire clk_dis;

```

```

1 assign val = (en_output ?
                output_data: val);
2 assign tmp_reg_a7 = (en_output ?
                       reg_a7: tmp_reg_a7);
3
4 SegGenerator uut_seg_gen (
5     ...
6 );
7
8 ClockDivider uut_clk_display (//
    2^16 bits for display
9     ...
10);
11
12 SegDisplay uut_seg_display (//
    display module
13     ...
14);
15 endmodule

```

Figure: Code of OutputModule

ClockDivider

```
1 module ClockDivider(  
2     input clk,  
3     input rst,  
4     input [31:0] period,  
5     output reg clk_out  
6 );  
7  
8 reg [24:0] cnt;
```

```
1 always @(posedge clk or negedge rst)  
2     begin  
3         if (~rst) begin  
4             cnt <= 0;  
5             clk_out <= 0;  
6         end else begin  
7             if (cnt == (period>>1)-1)  
8                 begin  
9                     cnt <= 0;  
10                    clk_out <= ~clk_out;  
11                end  
12            end else begin  
13                cnt <= cnt + 1;  
14            end  
15        end  
16 end  
17 endmodule
```

Figure: Code of ClockDivider

ClockDivider

```
1 module tb_ClockDivider;
2
3 reg      clk;
4 reg      rst;
5 reg [31:0] period;
6
7 wire      clk_out;
8
9 // Instantiate DUT
10 ClockDivider uut (
11     .clk      (clk),
12     .rst      (rst),
13     .period   (period),
14     .clk_out  (clk_out)
15 );
16
17 initial clk = 0;
18 always #5 clk = ~clk;
```

```
1 initial begin
2     $dumpfile("tb_ClockDivider.vcd");
3     $dumpvars(0, tb_ClockDivider);
4
5     rst      = 0;
6     period = 32'd20; // output clk
7                 toggles every (20/2)=10
8                 cycles -> 100 ns period
9
10    #10;
11    rst      = 1;
12    #500;
13    period = 32'd10; // output clk
14                 toggles every (10/2)=5
15                 cycles -> 50 ns period
16    #500;
17
18    $finish;
19 end
20 endmodule
```

Figure: Testbench of ClockDivider

ClockDivider

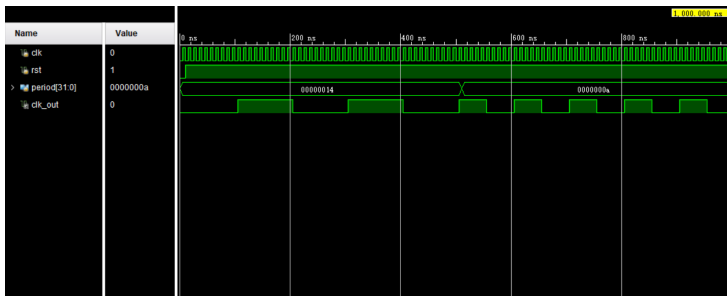


Figure: Waveform of ClockDivider

Clock Divider 波形解析

- 复位阶段: rst=0 时 clk_out 保持低; rst=1 后分频器开始工作
- 第一段 (period=20): 输出翻转间隔 10 个输入沿 $\rightarrow$ 半周期 $10 \times 5\text{ ns} = 50\text{ ns} \rightarrow$ 完整周期 100 ns
- 第二段 (period=10): 输出翻转间隔 5 个输入沿 $\rightarrow$ 半周期 $5 \times 5\text{ ns} = 25\text{ ns} \rightarrow$ 完整周期 50 ns

SegGenerator

```

1 module SegGenerator(
2     input rst,
3     input clk_dis,
4     output reg [3:0] select,
5     output wire [7:0] an
6 );
7 always @(posedge clk_dis or negedge
8     rst) begin
9     if (~rst) begin
10         select <= 4'd0;
11     end else begin
12         if (select == 4'd7) begin
13             select <= 4'd0;
14         end else begin
15             select <= select + 1;
16         end
17     end
18 end

```

```

1 assign an = (~rst) ? 8'b00000000 :
2     (select == 4'd0) ? 8'b10000000 :
3     (select == 4'd1) ? 8'b01000000 :
4     (select == 4'd2) ? 8'b00100000 :
5     (select == 4'd3) ? 8'b00010000 :
6     (select == 4'd4) ? 8'b00001000 :
7     (select == 4'd5) ? 8'b00000100 :
8     (select == 4'd6) ? 8'b00000010 :
9     (select == 4'd7) ? 8'b00000001 :
10     8'b11111111;
11 endmodule

```

Figure: Code of SegGenerator

SegGenerator

```
1 // 1. Declarations
2 module tb_SegGenerator;
3   // inputs
4   reg        rst;
5   reg        clk_dis;
6   // outputs
7   wire [3:0] select;
8   wire [7:0] an;
9 // 2. DUT instantiation
10 SegGenerator uut (
11   .rst      (rst),
12   .clk_dis  (clk_dis),
13   .select   (select),
14   .an       (an)
15 );
16 // 50 MHz clk_dis → toggle every 10
   ns
17 initial clk_dis = 0;
18 always #10 clk_dis = ~clk_dis;
```

```
1 // 3. Reset behavior
2 initial begin
3   rst = 0; #20;
4   rst = 1; #1;
5   if (select!=4'd0||an!=8'
       b10000000) $fatal("...");
6 // 4. Sequence through 10 ticks
7   integer i;
8   reg [3:0] expected_sel = 4'd0;
9   reg [7:0] expected_an  = 8'
       b10000000;
10  for (i=1;i<=10;i=i+1) begin
11    // wait one display-clock tick
12    // compute next expected and
       check DUT outputs
13  end
14  $display("...");
15  #10; $finish;
16 end
17 endmodule
```

Figure: Testbench of SegGenerator

SegGenerator

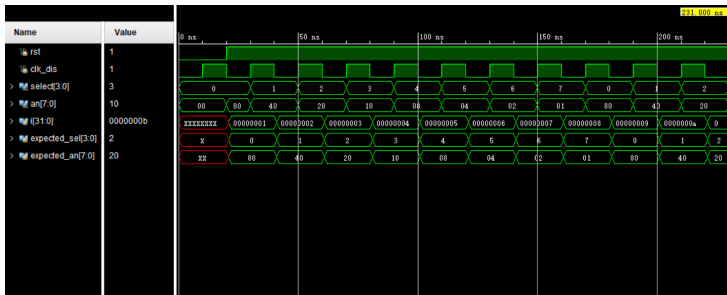


Figure: Waveform of SegGenerator

SegGenerator 波形验证

- 复位后: `select=0`, `an=8'b10000000`, 与 `expected` 相同
- 周期 1-8: 每个 `clk_dis` 上升沿, `select` 从 0 递增到 7, `an` 依次右移, 实际值与 `expected_sel/expected_an` 一致
- 周期 9-10: `select` 回到 0/1, `an` 回到 `8'b10000000/8'b01000000`, 匹配预期

SegDisplay

```
1 // 1. Header & ports
2 module SegDisplay(
3     input  [31:0] reg_a7,
4     input  [31:0] val,
5     input  [3:0] select,
6     output reg [7:0] led,
7     output reg [7:0] seg1,
8     output reg [7:0] seg2
9 );
10 // 2. Lookup table
11 reg [7:0] seg_map [0:15];
12 initial begin
13     // map hex digit → 7-segment
14     // pattern
15     seg_map[0]=8'b11111100;
16     ...
17     seg_map[15]=8'b10001110;
18     // (all 16 entries initialized)
19 end
```

```
1 // 3. Display logic
2 always @(*) begin
3     case (reg_a7)
4         32'd1: begin
5             led = 8'b00000000;
6             case (select)
7                 4'd0: seg1 = seg_map[(val
5                     /10000000) % 10];
8                 4'd1: seg1 = seg_map[(val
5                     /1000000) % 10];
9                 // ... digits 2-3 ...
10                4'd4: seg2 = seg_map[(val
5                    /1000) % 10];
11                // ... digits 5-7 ...
12                default: begin
13                    seg1 = 8'b00000000;
14                    seg2 = 8'b00000000;
15                    led = 8'b00000000;
16                end
17            endcase
18        end
```

Figure: Code of SegDisplay

SegDisplay (cont'd)

```
1  32'd1: begin
2    led = 8'b00000000;
3    case (select)
4      4'd0: seg1 = seg_map[val
           [31:28]];
5      4'd1: seg1 = seg_map[val
           [27:24]];
6      // ... digits 2-3 ...
7      4'd4: seg2 = seg_map[val
           [15:12]];
8      // ... digits 5-7 ...
9      default: begin
10         seg1 = 8'b00000000;
11         seg2 = 8'b00000000;
12         led = 8'b00000000;
13     end
14 endcase
15 end
```

```
1  32'd35: begin
2    // raw LED = low byte of val
3    seg1 = 8'b00000000;
4    seg2 = 8'b00000000;
5    led = val[7:0];
6    end
7    default: begin
8        // blank display
9        seg1 = 8'b00000000;
10       seg2 = 8'b00000000;
11       led = 8'b00000000;
12    end
13 endcase
14 end
15
16 endmodule
```

Figure: Code of SegDisplay (cont'd)

SegDisplay

```
1 module tb_SegDisplay;
2
3 // inputs
4 reg [31:0] reg_a7;
5 reg [31:0] val;
6 reg [3:0] select;
7
8 // outputs
9 wire [7:0] led;
10 wire [7:0] seg1;
11 wire [7:0] seg2;
12
13 // instantiate DUT
14 SegDisplay uut (
15     .reg_a7 (reg_a7),
16     .val (val),
17     .select (select),
18     .led (led),
19     .seg1 (seg1),
20     .seg2 (seg2)
21 );
```

```
1 initial begin
2 $dumpfile("tb_SegDisplay.vcd");
3 $dumpvars(0, tb_SegDisplay);
4 // ----- Test decimal display -----
5 reg_a7 = 32'd1;
6 val = 32'd12345678;
7 for (select = 0; select < 8; select
    = select + 1) begin
8     #1;
9     $display("DEC sel=%0d: seg1=%b
        seg2=%b led=%b", select, seg1
        , seg2, led);
10 end
11 // ----- Test hex display -----
12 ...
13 // ----- Test LED output -----
14 ...
15
16 #1 $finish;
17 end
18 endmodule
```

Figure: Testbench of SegDisplay

SegDisplay

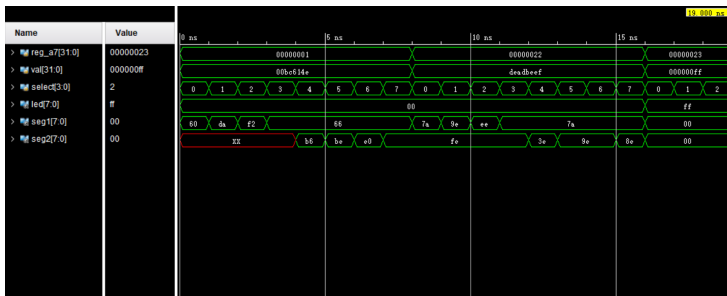


Figure: Waveform of SegDisplay

SegDisplay 波形验证

- 模式 1 (`reg_a7=1`): `select` 0-3 时 `seg1` 映射十进制高位, 4-7 时 `seg2` 映射十进制低位; `led=8'b00000000`
- 模式 34 (`reg_a7=34`): `select` 0-7 依次选择 `val` 的 8 个半字节, `seg1/seg2` 分别输出当前半字节的八段码; `led=8'b00000000`
- 模式 35 (`reg_a7=35`): `led` 直接输出 `val[7:0]`, `seg1` 与 `seg2` 均为 `8'b00000000`

ALU

```
1 // 1. Declarations
2 module ALU(
3     input  [31:0] read_data1,
4     input  [31:0] read_data2,
5     input  [31:0] imm32,
6     input          alu_src,
7     input  [1:0]  alu_op,
8     input  [2:0]  funct3,
9     input  [6:0]  funct7,
10    output reg [31:0] alu_result,
11    output reg      zero,
12    input  is_lui,
13    input  is_auipc,
14    input  [31:0] pc
15);
16
17 wire [31:0] operand2;
18 wire [4:0] shamt; // shift amount (
    only low 5 bits valid for 32-
    bit)
```

```
1 // 2. Operand select
2 wire [31:0] operand2 = alu_src ?
    imm32 : read_data2;
3 wire [4:0] shamt = operand2
    [4:0]; // shift amount
4
5 always @(*) begin
6     case (alu_op)
7         2'b00: begin
8             // Load/Store:
9             // alu_result = read_data1 +
                operand2
10        end
```

Figure: Code of ALU

ALU (cont'd)

```
1 // 3. Branch (2'b01)
2   2'b01: begin
3       // subtract and set zero flag:
4       // BEQ/BNE: eq vs neq
5       // BLT/BGE: signed compare
6       // BLTU/BGEU: unsigned compare
7   end
8
9 // R-type (2'b10)
10  2'b10: begin
11      // use funct3/funct7 to select:
12      // ADD/SUB, XOR, OR, AND,
13      // SLL, SRL, SRA, SLT, SLTU
14  end
```

```
1 // 4. I-type & U-type (2'b11)
2   2'b11: begin
3       // if is_lui: alu_result =
4           operand2
5       // else if is_auipc:
6           alu_result = pc +
7               operand2
8       // else: perform ADDI, XORI,
9           ORI, ANDI,
10          SLLI, SRLI, SRAI, SLTI,
11          SLTIU
12   end
13   default: alu_result = 32'd0; //
14       safe default
15 endcase
16 end
17 endmodule
```

Figure: Code of ALU (cont'd)

ALU

```
1 // 1. Declarations
2 module tb_ALU;
3   // inputs
4   reg [31:0] read_data1, read_data2,
        imm32, pc;
5   reg alu_src, is_lui, is_auiopc;
6   reg [1:0] alu_op;
7   reg [2:0] funct3;
8   reg [6:0] funct7;
9   // outputs
10  wire [31:0] alu_result;
11  wire zero;
```

```
1 // 2. DUT instantiation
2 ALU uut (
3   .read_data1(read_data1),
4   .read_data2(read_data2),
5   .imm32      (imm32),
6   .alu_src    (alu_src),
7   .alu_op     (alu_op),
8   // ... other port connections ...
9   .alu_result(alu_result),
10  .zero       (zero)
11 );
```

Figure: Testbench of ALU

ALU (cont'd)

```
1 // 3. Tests 1-3
2 initial begin
3     // waveform dump setup
4     $dumpfile("tb_ALU.vcd");
5     $dumpvars(0, tb_ALU);
6
7     // Test 1: ADD (load/store)
8     //   alu_src=0; alu_op=00;
9     //   read_data1=15; read_data2=27;
10    //   expect alu_result==42
11
12    // Test 2: BEQ zero detection
13    //   alu_op=01; funct3=000; data1
14    //   ==data2;
15    //   expect zero==1
16
17    // Test 3: R-type SLL
18    //   alu_op=10; funct3=001; shift
19    //   left by shamt
```

```
1 // 4. Tests 4-7 & cleanup
2 // Test 4: R-type SRA (arithmetic
3 //   right shift)
4 // Test 5: I-type ANDI (imm vs reg
5 //   )
6 // Test 6: LUI (is_lui=1; expect
7 //   operand2)
8 // Test 7: AUIPC (is_auiipc=1;
9 //   expect pc+operand2)
10
11 $display("All ALU tests passed");
12 #1; $finish;
13 end
14
15 endmodule
```

Figure: Testbench of ALU (cont'd)

ALU

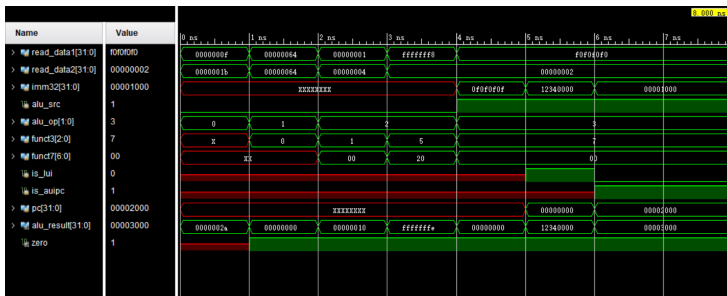


Figure: Waveform of ALU

ALU 波形验证

- Test 1 (`alu_op=2'b00`, `alu_src=0`): 加法 → `alu_result=0x0000002A`, `zero=0`
- Test 2 (`alu_op=2'b01`): 分支减法 → `zero=1`
- Test 3 (`alu_op=2'b10`, `funct3=3'b001`): 逻辑左移
- Test 4 (`alu_op=2'b10`, `funct3=3'b101`, `funct7[5]=1`): 算术右移 → `alu_result=0xFFFFFFF`

ALU 波形验证 (cont'd)

- Test 5 (alu_op=2'b11, funct3=3'b111): ANDI →
alu_result=0x00000000
- Test 6 (is_lui=1): LUI → alu_result=0x12340000
- Test 7 (is_auiipc=1): AUIPC →
alu_result=pc+imm=0x00003000

经核查上述运算结果均正确。

THANK YOU!